LEDs remotos

Conexiones y configuración del servidor.

Introducción

Vamos a montar un pequeño servidor que controlará a unos leds. La secuencia para encender o apagar los leds se gestionará de manera remota con una aplicación web.

Vamos a usar:

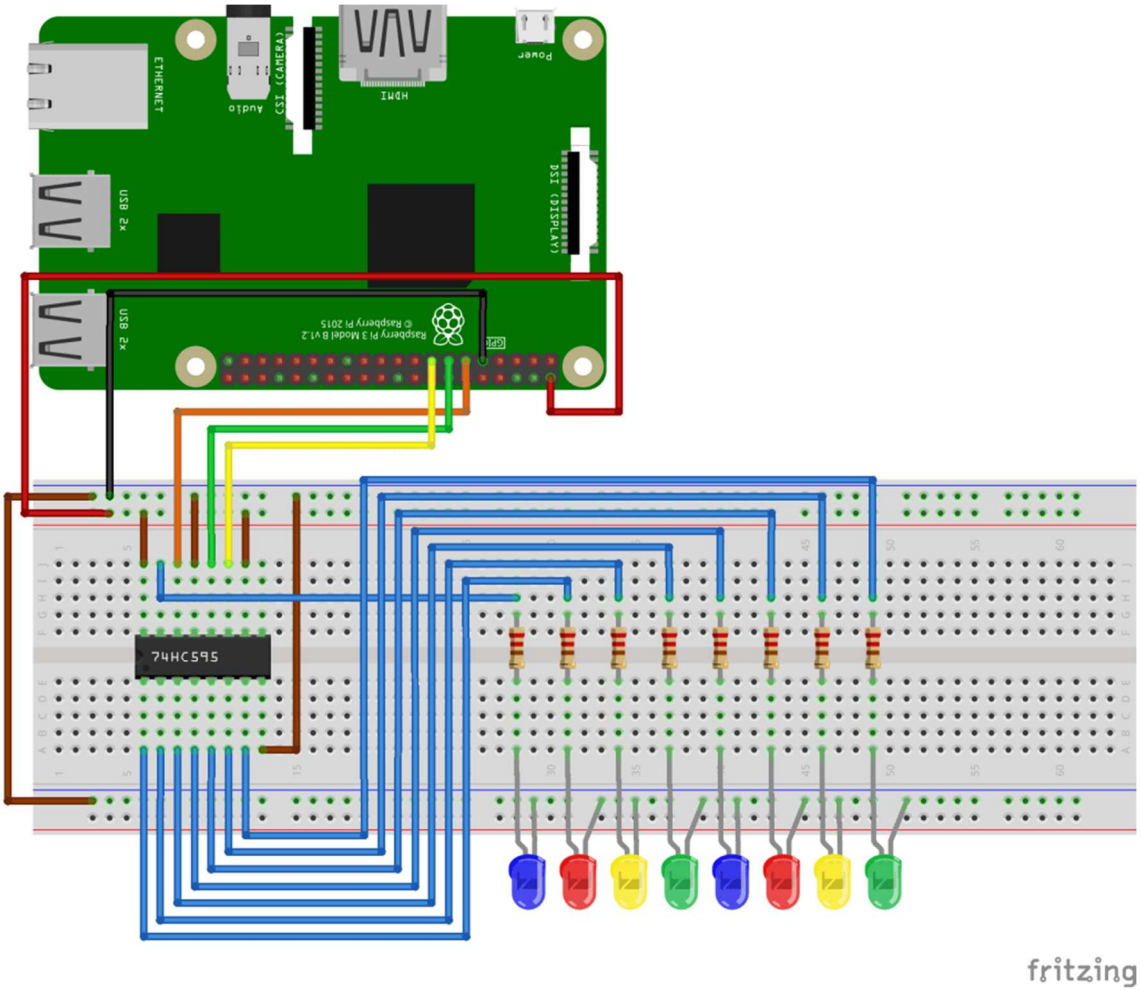
- **Python** (versión 3) para el control de los leds conectados a la GPIO.
- **Apache** (versión 2) y **PHP** (versión 7) como tecnología back-end para ejecutar los scripts de Python.
- **Angular** (versión 7) como tecnología Front-end para la aplicación para el control de los leds.

Preparación de la Raspberry Pi, que será la encargada de controlar los leds, con el servidor web, configuraciones y conexiones necesarias.

Componentes

- Raspberry Pi
- 8 leds (2 azul, 2 rojo, 2 amarillo, 2 verde)
- 8 resistencias de 220 ohms
- 1 Circuito integrado SN74HC595N
- Protoboard y cables

Esquema (Fritzing)



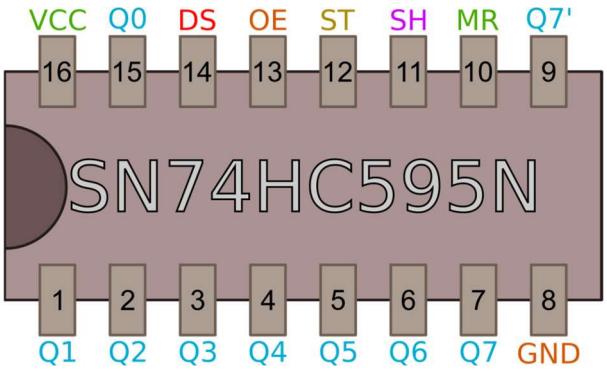
El circuito integrado SN74HC595N

El circuito integrado SN74HC595N es un cambiador de registro, o *shift register*. Admite la entrada de datos en series y los devuelve a una salida paralela, en 8 pins. Para manejar un SN74HC595N sólo hacen falta tres pines de la GPIO y podemos controlar múltiples leds, ya que, además, se pueden conectar varios SM74HC595N en cascada.

La funcionalidad de cada pin es la siguiente:

- **Data**: se usa para la entrada de datos.
- **Clock**: se usa para indicar cuando se ha de leer el valor recibido en el pin Data
- **Latch**: se usa para memorizar y enviar los datos recibidos por el pin Data hacia la salida.

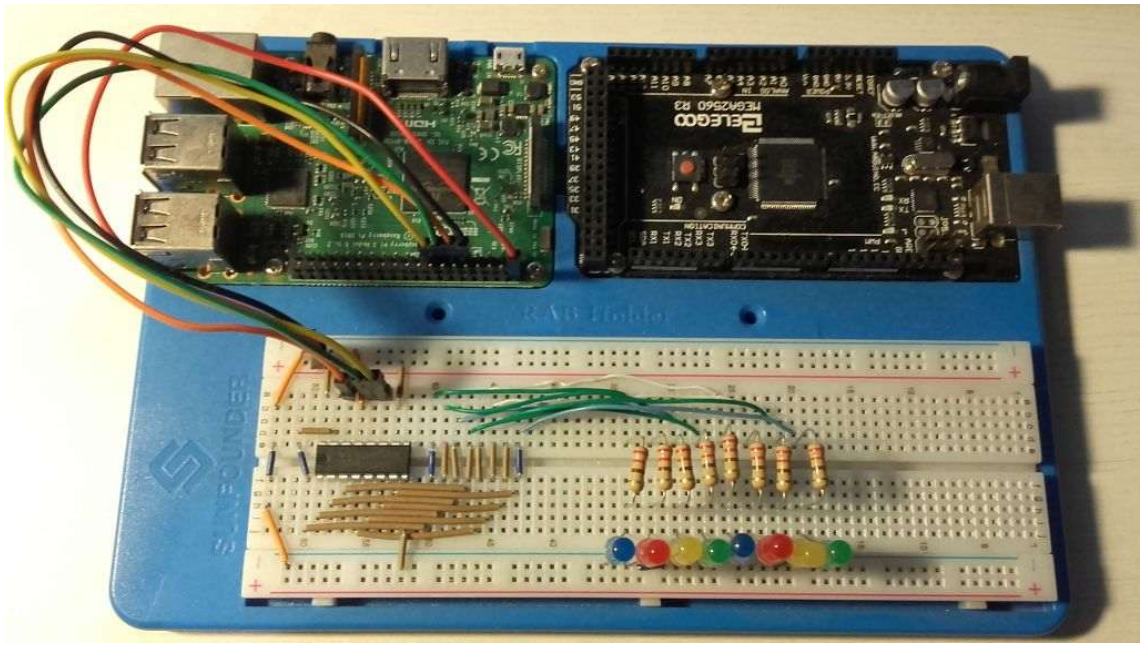
Conexiones del SN74HC595N

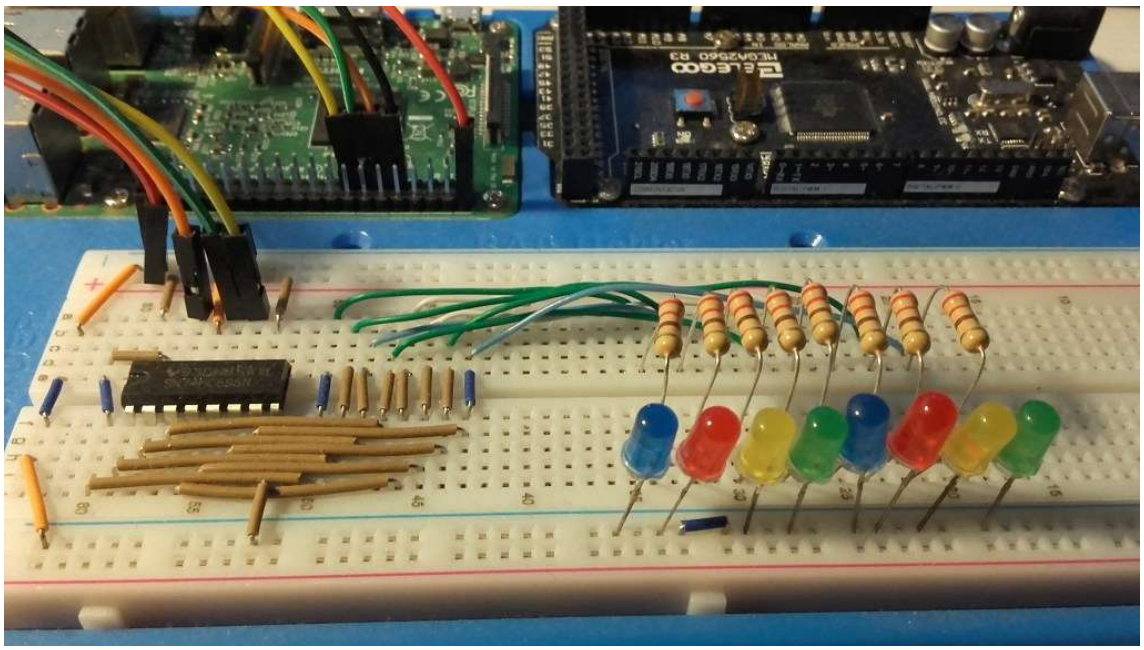


Las conexiones serán las siguientes:

Patilla	**Conectado a... / Descripción**
1 (Q1) Salida	Al segundo led (rojo)
2 (Q2) Salida	Al tercer led (amarillo)
3 (Q3) Salida	Al cuarto led (verde)
4 (Q4) Salida	Al quinto led (azul)
5 (Q5) Salida	Al sexto led (rojo)
6 (Q6) Salida	Al séptimo led (amarillo)
7 (Q7) Salida	Al octavo led (verde)
8 (GND)	Conectado a masa
9 (Q7') Salida	Permite conectar otro integrado en cascada. En este caso no lo vamos a usar.
10 (MR) Master reset	Conectado a 5V Permite resetear los valores almacenados al recibir un valor bajo.

	Lo mantenemos en alto para que no actue.
11 (SH) Clock	Pin 15 de la GPIO
12 (ST) Latch	Pin 13 de la GPIO
13 (OE) Output enabled	Permite habilitar las salidas del integrado. Se activa al recibir un valor bajo. Lo conectamos a masa.
14 (DS) Data	Pin 11 de la GPIO
15 (Q0) Salida	Al primer led (azul)
16 (VCC)	Conectado a alimentación, 5V.





Instalación de Apache

Para instalar Apache usaremos apt-get. Primero se han de actualizar las listas de paquetes, mediante el comando:

```
sudo apt-get update
```

Una vez actualizadas las listas, ejecutamos el siguiente comando para instalar Apache:

```
sudo apt-get install apache2 -y
```

Para comprobar si se ha instalado correctamente, basta con acceder, usando un navegador, a **http://localhost**, desde la Raspberry, o, si lo hacemos desde otro dispositivo de la misma red, accediendo a **http://[IP DE LA RASPBERRY]**.

Aparecerá la página por defecto:



Cambios en el directorio raíz de Apache

Para que el servidor no rechace las peticiones que se harán desde la aplicación de Angular, que se servirá desde su propio servidor, se han de habilitar las solicitudes de recursos de origen cruzado o CORS (https://es.wikipedia.org/wiki/Intercambio_de_recursos_de_origen_cruzado). Se puede hacer creando un fichero .htaccess en la raíz del sitio o editando el fichero **/etc/apache2/sites-available/000-default.conf**. Editamos el fichero fichero usando nano:

```
sudo nano /etc/apache2/sites-available/000-default.conf
```

Buscamos la línea donde se define DocumentRoot y, dentro de la configuración del directorio raíz añadimos lo siguiente:

```
Header add Access-Control-Allow-Origin "*"
```

El contenido del fichero quedaría así:

```
DocumentRoot /var/www/html
<Directory /var/www/html>
          Require all granted
          Header add Access-Control-Allow-Origin "*"
</Directory>
```

El siguiente cambio es opcional. Se trata de cambiar la ubicación del directorio donde se alojan los ficheros que se mostrarán cuando se haga un petición a Apache. Sin salir del fichero que estamos editando, modificamos la ruta de DocumentRoot por la del directorio que aloje los ficheros a servir. Por ejemplo:

DocumentRoot /home/usuario/servidorWeb

<Directory /home/usuario/servidorWeb>

...

</Directory>

Finalmente, se ha de reiniciar el servidor de Apache:

```
sudo service apache2 restart
```

Instalación de PHP

Para instalar PHP, volvemos a usar apt-get, esta vez con el siguiente comando:

```
sudo apt-get install php libapache2-mod-php -y
```

Reiniciamos el servidor de Apache y probamos la instalación de PHP creando un fichero index.php con la siguiente línea:

```
<?php phpinfo(); ?>
```

Si estamos usando el directorio raíz por defecto de Apache, se ha de renombrar o borrar el fichero index.html existente para que Apache no lo tenga en cuenta al hacer la petición desde el navegador. Al acceder debería mostrar la página de información de PHP.

Últimos cambios en el servidor

Por último, modificaremos los derechos del directorio donde se alojan los ficheros para dar derechos de lectura y ejecución para el propietario, grupo y otros, y de escritura para el propietario. También añadiremos al usuario **www-data** a los usuarios a **sudoers**, para que pueda ejecutar los scripts de Python.

Para cambiar los derechos del directorio usaremos el siguiente comando:

```
sudo chmod 755 [directorio]
```

Para añadir el usuario **www-data** a sudoers podemos usar la utilidad **visudo**. Ejecutamos:

```
sudo visudo
```

Se abrirá un editor de texto donde añadiremos lo siguiente:

```
www-data ALL=(ALL) NOPASSWD: ALL
```

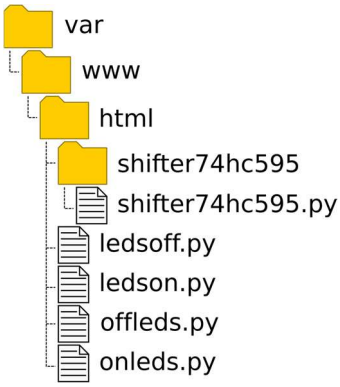
Esta modificación permitirá al usuario www-data ejecutar los scripts de Python como sudo sin tener que introducir la contraseña. Pulsamos Ctrl+X y aceptamos los cambios para guardar.

Back-end: Control de los leds mediante Python y PHP.

Introducción

Vamos a programar un módulo de Python para controlar el integrado SN74HC595 y dos scripts para apagar y encender los leds. También vamos a programar dos scripts de PHP que se encargaran de recibir las peticiones de la aplicación de gestión de los leds y ejecutar los scripts de Python.

En caso de usar el directorio por defecto de Apache para alojar ficheros, la estructura de ficheros, quedaría así:



Scripts de Python

El módulo shifter74hc595

Para el manejo del SN74HC595N vamos a crear un módulo de Python para usar tanto en esta aplicación como en posteriores que podamos realizar. Un módulo es un archivo que contiene funciones, clases, … que importaremos en otros ficheros para su uso. En este caso, el módulo contendrá una clase.

Esta clase contendrá una matriz donde se guardarán los estados (HIGH o LOW) de los leds en un momento determinado. La información de esta matriz se enviará al integrado cuando se quiera encender o apagar los leds, según convenga.

Para poder manejar los leds necesitaremos el módulo RPi.GPIO, lo importamos

```
import RPi.GPIO as GPIO
```

Declaramos la clase y cuatro propiedades de esta: tres para los pines de datos, reloj y fijar, y la matriz para guardar el estado de los 8 pines.

```
class shifter74hc595():
data = None;
clock = None;
latch = None;
leds = [None, None, None, None, None, None, None, None]
```

El método __init__

El método que inicializa el objeto al instanciarlo recibirá como parámetros los tres pins para los datos, reloj y fijar, y se encargará de asignar estos valores a sus propiedades, inicializar la GPIO (en modo BOARD) y los pines (en modo OUT).

También ejecutará un par de métodos (se verán más adelante) para establecer todos los pins con un valor bajo (apagados).

```
def __init__(self, data, clock, latch):
          self.data = data
          self.clock = clock
          self.latch = latch
          GPIO.setmode(GPIO.BOARD)
          GPIO.setwarnings(False)
          GPIO.setup(self.data, GPIO.OUT)
          GPIO.setup(self.clock, GPIO.OUT)
          GPIO.setup(self.latch, GPIO.OUT)
          self.setAll(GPIO.LOW)
          self.sendData()
```

El método __del__

Añadimos un método __del__, para que cuando se destruya el objeto, apague los leds y limpie la configuración de la GPIO establecida en el método de inicio.

```
def __del__(self):
          self.setAll(GPIO.LOW)
          self.sendData()
          GPIO.cleanup()
```

El método setLed

Este método permite establecer el valor para uno de los leds en concreto, representado por su índice en la matriz. Recibe como parámetros el índice del led a modificar y el valor.

```
def setLed(self, led, value):
          self.leds[led] = value
```

El método setAll

Este método permite establecer un valor común para todos los leds. El parámetro recibido es el valor a aplicar a los leds.

```
def setAll(self, value):
          for i in range(len(self.leds)):
                    self.leds[i] = value
```

El método setOnLed

Este método permite encender led, modificando su estado. Recibe como parámetro el índice del led a modificar.

```
def setOnLed(self, led):
          self.setLed(led, GPIO.HIGH)
```

El método setOffLed

Este método permite apagar led, modificando su estado. Recibe como parámetro el índice del led a modificar.

```
def setOffLed(self, led):
          self.setLed(led, GPIO.LOW)
```

El método __setDataPin

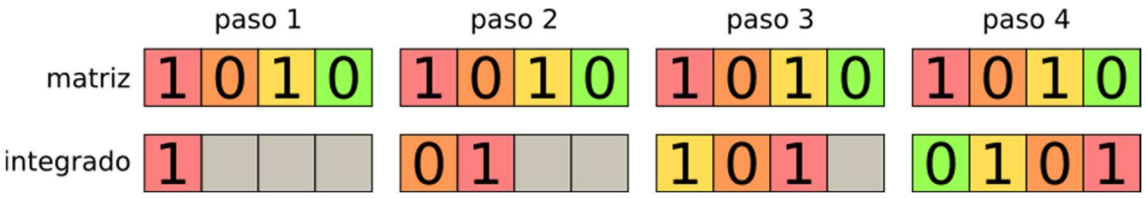
Este método envía un estado para la posición actual en la entrada de datos del integrado. Para esto envía un valor bajo al reloj para desactivarlo, posteriormente envía el estado que ha recibido como parámetro (HIGH o LOW) y, finalmente, pone en alto al reloj para que el dato enviado se inserte en la lista.

```
def __setDataPin(self, state):
          GPIO.output(self.clock, GPIO.LOW)
          GPIO.output(self.data, state)
          GPIO.output(self.clock, GPIO.HIGH)
```

El método sendData

Este método envía los datos alojados en la matriz de estados al integrado. Para esto, pone el pin para fijar en bajo y envía los estados al integrado. Cuando ya se han enviado todos activa el pin para fijar, de manera que se envían los datos a la salida paralela.

Se invierte el orden de la matriz ya que, al enviar un valor, este *empuja* al anteriormente enviado, quedando, una lista como la enviada pero en orden inverso.



Ejemplo de como se distribuyen los valores de una matriz según se van enviando al integrado.

```
def sendData(self):
          GPIO.output(self.latch, GPIO.LOW)
          for i in reversed(self.leds):
                    self.__setDataPin(i)
                    GPIO.output(self.latch, GPIO.HIGH)
```

Final del módulo shifter74hc595

El módulo siempre se va a importar desde otro script, por lo que no es necesario que haga nada en caso de ser ejecutado desde la línea de comandos. Para esto, incluimos al final del fichero una comprobación de la variable especial *__name__*. Si su valor es *__main__*, significa que se ha ejecutado desde la línea de comando, por lo que llamamos a *pass* para que no haga nada.

```
if __name__ == "__main__":
        pass
```

El script ledson.py

Este script se encarga de encender y apagar los leds según la información de la secuencia recibida desde la aplicación de gestión.

Como primer paso, importamos el módulo para el manejo del SN74HC595 y algunos otros:

- **sys**: nos da funcionalidades y variables respecto al interprete de Python.
- **os**: nos da funcionalidades y variables respecto al sistema operativo.
- **time**: nos da funcionalidades para el manejo de fechas, horas y tiempo.
- json: nos da funcionalidades para el manejo de datos en formato JSON.

También definimos una variable a la que le asignamos un objeto de la clase **shifter74hc595**, pasándole como parámetros los tres pines para datos, reloj y fijar.

Para finalizar, añadimos una comprobación de la variable especial *__name__*. Si es igual a *__main__* (se esta ejecutando desde la línea de comandos), ejecutamos las ordenes necesarias para encender y apagar los leds según la secuencia.

Dentro de la condición de __*name*__ vamos a realizar lo siguiente:

Obtendremos, mediante al módulo os, el id del proceso que se ha generado al ejecutar el script y lo guardaremos en un fichero con el nombre *leds.pid.txt*, esto lo usaremos en el momento de apagar los leds.

Abrimos el fichero *data.json*, que contiene los datos sobre la secuencia a ejecutar, y, mediante el módulo json, creamos un objeto de Python con los datos.

Creamos un bloque try / except, para poder parar la ejecución del script mediante una interrupción de teclado (Ctrl + C), al ejecutar el script desde la línea de comandos.

Dentro del try, creamos un bloque While infinito donde recorremos el contenido de la propiedad *steps* del objeto, una matriz que contiene la información de estado de los leds en cada paso de la secuencia.

Por cada paso, recorremos la matriz guardada en la propiedad *states*, que contiene el estado de cada uno de los leds, y lo establecemos en la matriz del objeto *shifter74hc595* mediante *setLed*, para, al finalizar, enviarlo al integrado con *sendData*.

Finalmente, usando la función *sleep* de time, hacemos una pausa que durará lo que indique la propiedad *sleep* del paso que se esta ejecutando.

```
#!/usr/bin/env python3
from shifter74hc595.shifter74hc595 import shifter74hc595
import sys
import os
from time import sleep
import json

shifter = shifter74hc595(11, 15, 13)

if __name__ == '__main__':
          # get the pid
          pid = os.getpid()
          file = open('leds.pid.txt', 'w')
          file.write(str(pid))
          file.close()

          # open json data file
          with open('data.json', 'r') as file:
                    leds_steps = json.load(file)

          try:
                    while True:
                              for step in leds_steps['steps']:
                                        for idx, state in enumerate(step['states']):
                                                  shifter.setLed(idx, state)
                                        shifter.sendData()
                                        sleep(float(step['sleep']))
          except KeyboardInterrupt:
                    del(shifter)
                    sys.exit()
```

El script ledsoff.py

Este script se encarga de apagar todos los leds.

Se importan los módulos shifter74hc595 y sys.

Después de comprobar si la variable especial *__name__* contiene *__main__*, se establecen todos los estados de los leds a LOW, mediante *setAll*, se envían al integrado y se sale del script.

```
#!/usr/bin/env python3
from shifter74hc595.shifter74hc595 import shifter74hc595
import sys

shifter = shifter74hc595(11, 15, 13)

if __name__ == '__main__':
          shifter.setAll(0)
          shifter.sendData()
          del(shifter)
          sys.exit()
```

Scripts de PHP

El script onleds.php

Este script se encargará de recibir la información de la secuencia de leds desde la aplicación, escribir un fichero JSON con la información y llamar al script de Python.

Si al realizar alguna de las acciones se produce un error, se devolverá a la aplicación un objeto JSON con dos propiedades: una con nombre *result* y valor *false*, y otra, con nombre *message* y, como valor, el motivo del error.

Si todo ocurre correctamente, se devolverá a la aplicación un objeto JSON con una propiedad de nombre *result* y con un valor *true*.

En un primer paso comprueba si se han recibido los datos vía POST y se crea una matriz con un campo que contiene la información recibida.

A continuación se intenta guardar la información en un fichero JSON, que será el que abrirá, posteriormente, el script de Python.

Finalmente se ejecuta el script de Python, enviando la salida del script a segundo plano, de manera que el script de PHP finalice y envíe el resultado a la aplicación.

```php
<?php
// gets, from the received post data, the information to play the sequence
if(!array_key_exists('data', $_POST)) {
	die(json_encode(array('result' => false, 'message' => 'no data key')));
} else {
	$data = $_POST['data'];
	$steps = json_decode($_POST['data']);
	$objectForFile = array('steps' => $steps);
}

// saves the information into a json file, this file is used
// by the python script that play the sequence
try {
	$fp = fopen('data.json', 'w');
	if(!$fp) {
		die(json_encode(array('result' => false, 'message' => 'error encoding
data')));
	}
	$result = fwrite($fp, json_encode($objectForFile));
	if(!$result) {
		die(json_encode(array('result' => false, 'message' => 'error saving
data')));
	}
	fclose($fp);
} catch(Exception $e) {
	// exception when trying to save JSON file with data
	die(json_encode(array('result' => false, 'message' => $e->getMessage())));
}

// execute the python script,
try {
	$command = 'sudo python3 ledson.py >/dev/null &';
	system($command, $result);
	if($result > 0) {
		die(json_encode(array('result' => false, 'message' => 'error on turn on
leds')));
	}
} catch(Exception $e) {
	// exception when trying to execute python script
	die(json_encode(array('result' => false, 'message' => $e->getMessage())));
}

// all OK, sends true to front-end
echo json_encode(array('result' => true));
?>
```

El script offleds.php

Este script se encargará de parar la reproducción de la secuencia de leds. Al igual que el script anterior, devuelve un objeto JSON, con el resultado.

Primero intentar abrir el fichero *leds.pid.txt*, que contiene el identificador del proceso que esta ejecutando el script de Python *ledson.py*, y leer su contenido.

En caso de obtener el contenido del fichero, lo usa para finalizar el proceso, mediante el comando *kill* del sistema.

Por último, ejecuta el script de Python *ledsoff.py* para apagar todos los leds. Así, si algún led estaba encendido en el momento de matar el proceso de reproducción de la secuencia, quedará apagado.

```php
<?php
// opens the file that contains the process id
// of python script that is running the leds sequence
try {
          $pidfile = 'leds.pid.txt';
          $fp = fopen($pidfile, 'r');
          if(!$fp) {
                    die(json_encode(array('result' => false, 'message' => 'error getting
pid')));
          }
          $pid = fread($fp, filesize($pidfile));
          fclose($fp);
} catch (Exception $e) {
          die(json_encode(array('result' => false, 'message' => $e->getMessage())));
}

// it executes a command to kill the process
try {
          $command = 'sudo kill -9 ' . $pid;
          system($command, $result);
          if($result > 0) {
                    die(json_encode(array('result' => false, 'message' => 'error on kill
process')));
          }
} catch (Exception $e) {
          die(json_encode(array('result' => false, 'message' => $e->getMessage())));
}

// it executes a python script that get off all the leds
try {
          $command = 'sudo python3 ledsoff.py';
          system($command, $result);
          if($result > 0) {
                    die(json_encode(array('result' => false, 'message' => 'error on turn
off leds')));
          }
} catch (Exception $e) {
          die(json_encode(array('result' => false, 'message' => $e->getMessage())));
}
echo json_encode(array('result' => true));
?>
```

Front-end: Aplicación de Angular para gestionar las secuencias

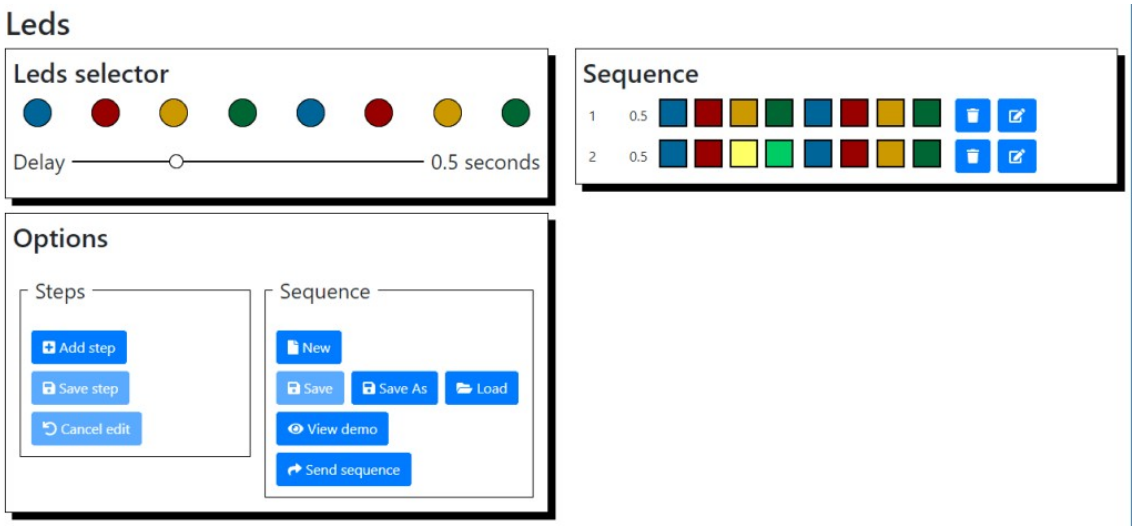
Introducción

Vamos a ver una aplicación programada con Angular para la gestión de las secuencias. Se podrán crear, editar, guardar, cargar las secuencias, además de enviar a la Raspberry peticiones para gestionar su reproducción.

Aspecto y funcionamiento de la aplicación

Pantalla principal

Al entrar en la aplicación se muestra la siguiente pantalla:



Esta dividida en tres secciones:

- **Leds selector**: donde podremos definir el estado de los leds para un paso en la secuencia y el tiempo que se mantendrán los leds en ese estado.
- **Sequence**: un listado con los pasos que vayamos añadiendo a la secuencia. Cada paso dispone de dos opciones: *borrar* y *editar*.
- **Options**: una serie de opciones para ejecutar diferentes acciones como añadir, editar, guardar, ... sobre los pasos y secuencias.

La sección de opciones, a su vez, tiene agrupados sus botones en dos secciones:

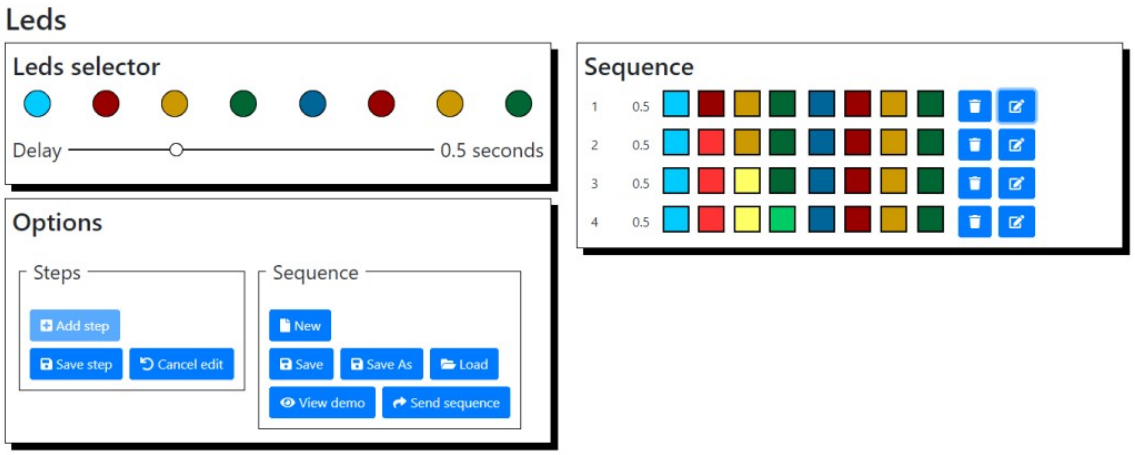
- **Steps**: acciones relacionadas con los pasos de la secuencia:
    - **Add step**: añade a la secuencia un paso con la configuración de estados actual mostrada en la sección Leds selector.
    - **Save step**: guarda la información de un paso de la secuencia que se esté editando.
    - **Cancel edit**: cancela la edición de un paso, sin modificar la información de la secuencia.

- **Sequence**: acciones relacionadas con la gestión de las secuencias.
    - **New**: permite crear una nueva secuencia.
    - **Save**: guarda la información de una secuencia en el local storage del navegador. Para que este disponible, previamente, se deberá haber guardado la secuencia usando **Save As**.
    - **Save As**: guarda la información de una secuencia en el local storage del navegador asignándole un nombre.
    - **Load**: abre una pantalla donde podremos cargar una de las secuencias guardadas en el local storage.
    - **View demo**: permite ver una demostración de la secuencia, sin necesidad de enviarla a la Raspberry.
    - **Send sequence**: Envía la secuencia a la Raspberry para su reproducción.

Editar una secuencia

Al pulsar sobre el botón **Editar** de cualquiera de uno de los pasos de la secuencia se cargan los datos en el Leds selector y se activan los botones **Save step** y **Cancel edit**.

Save step actualizará los datos del paso en la secuencia y **Cancel edit** cancelará la acción sin realizar ningún cambio en la secuencia.

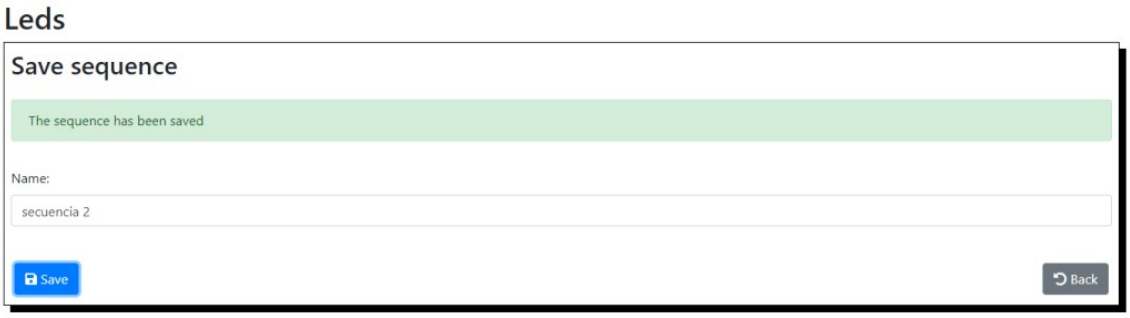


Guardar secuencia

En la pantalla para guardar secuencias disponemos de un campo de texto para indicar el nombre con el que la queremos guardar y dos botones. **Save**, que nos permitirá guardar al secuencia en el local storage del navegador, y **Back**, que nos devolverá a la pantalla principal.



Pantalla de guardar al acceder



Pantalla de guardar mostrando el mensaje que la operación se ha finalizado correctamente

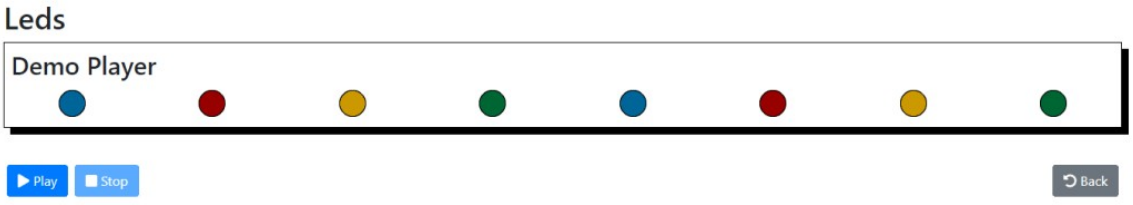
Cargar secuencia

La pantalla para cargar secuencia muestra una lista con las secuencias disponibles en el local storage. Al pulsar sobre alguna de ellas se activa el botón **Load**. Al pulsarlo, se cargará la información de la secuencia y se volverá a la pantalla inicial. El botón **Back** vuelve a la pantalla inicial sin realizar acción alguna.



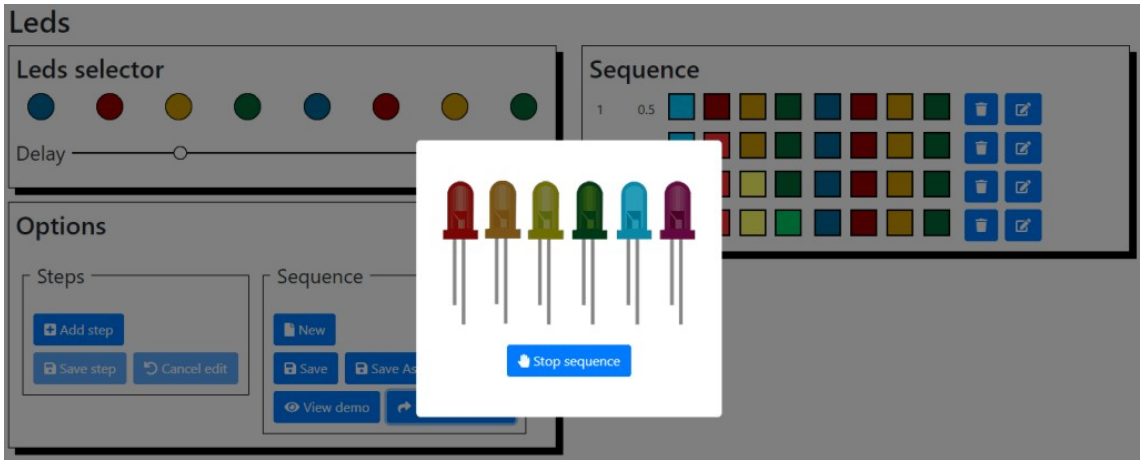
Reproducción demostración

La pantalla del reproductor de secuencias muestra unas imágenes a modo de LEDs y dos botones, **Play** y **Stop**, para controlar la reproducción de la demostración. También dispone de un botón **Back** para volver a la pantalla principal.



Reproduciendo la secuencia

Al enviar una secuencia a reproducir en la Raspberry se muestra un pop-up con un botón para parar la reproducción.



En caso de producirse un error, se muestra un mensaje:



Componentes de la aplicación

La aplicación, al igual que cualquier otra creada con Angular, está formada por componentes, servicios, interfaces, ...

Componentes

leds-root

Es el componente principal. Contiene un componente para mostrar el título de la aplicación y contenedor para las vistas del router.

leds-title

Este componente muestra títulos en la aplicación. Tanto de la aplicación en sí como de las diferentes opciones: guardar, cargar, ...

Recibe dos parámetros:

- **size**: el tamaño del título, pudiendo ser grande, mediano o pequeño.
- **text**: el texto a mostrar en el título

leds-home

Este componente es el contenedor de todos los que forman la pantalla principal de la aplicación. Usa los siguientes componentes:

- **leds-container**: para mostrar los leds con los que definir el paso a añadir a la secuencia.
- **leds-options**: para mostrar las opciones para los pasos y secuencias.
- **leds-displayer**: para mostrar el listado de pasos de la secuencia y sus opciones.
- **leds-pop-up**: para mostrar un pop-up con mensajes y preguntas de confirmación al usuario.
- **leds-loader**: para mostrar el *loading* y el pop-up con el resultado cuando se envía una secuencia a la Raspberry.

leds-container

Este componente muestra todos los leds de un paso y, si así lo indica el parámetro correspondiente, la barra para definir el tiempo que durará el paso.

Se usa en la pantalla principal, para configurar los pasos de la secuencia, y en la pantalla de reproducción de demostración, para mostrar cómo se mostrarán los leds y sus estados durante la reproducción (en este caso no muestra la barra para definir la duración del paso).

Usa los siguientes componentes:

- **leds-title**: para mostrar el título de la sección.
- **leds-led**: este componente representa un led. Leds-container usa 8 para representar los 8 leds conectados a la Raspberry.

Recibe dos parámetros:

- **title**: el título de la sección. El componente lo pasará como parámetro a leds-title.
- **showDelayBar**: si debe mostrar o no la barra que define la duración del paso.

leds-pop-up

Este componente muestra un pop-up con un mensaje informativo o confirmación a una acción, dependiendo de un parámetro.

Recibe tres parámetros:

- **title**: el título del pop-up
- **message**: el mensaje del pop-up
- **type**: el tipo de pop-up a mostrar, entre alerta o confirmación

Dispone también de dos eventos:

- **close**: se dispara cuando se ha de cerrar el pop-up, mediante el botón cerrar de los mensajes de alerta o el botón cancelar de los mensajes de confirmación.
- **confirm**: se dispara cuando el usuario acepta un mensaje de confirmación.

leds-loader

Este componente muestra el pop-up que informa del resultado de enviar la secuencia a la Raspberry.

Recibe como parámetro:

- **loader**: una promesa sobre la petición a la Raspberry para reproducir la secuencia. La usa para mostrar el loader mientras esta cargando y esconderlo, mostrando el resultado, cuando finaliza.

Dispone de un evento:

- **close**: para cerrar el pop-up.

leds-options

Este componente muestra las opciones para los pasos y la secuencia. Usa el siguiente componente:

- **leds-title**: para mostrar el título de la sección.

leds-displayer

Este componente muestra la lista de pasos que se va añadiendo a la secuencia según se define. Usa el siguiente componente:

- **leds-title**: para mostrar el título de la sección.

Dispone de un evento:

- **delete**: se dispara al borrar un paso, para que el componente padre (leds-home) muestre un pop-up para pedir confirmación.

leds-demo-player

Este componente es el encargado de mostrar la pantalla para reproducir la demostración de la secuencia. Usa el siguiente componente:

- **leds-container**: para mostrar los leds con los que definir el paso a añadir a la secuencia.

leds-save

Este componente muestra la pantalla para guardar una secuencia en el local storage del navegador. Usa los siguientes componentes:

- **leds-title**: para mostrar el título de la pantalla.
- **leds-messages-box**: para informar al usuario sobre incidencias que se puedan producir al ejecutar la acción.

leds-load

Este componente muestra la pantalla para cargar una secuencia del local storage del navegador. Usa los siguientes componentes:

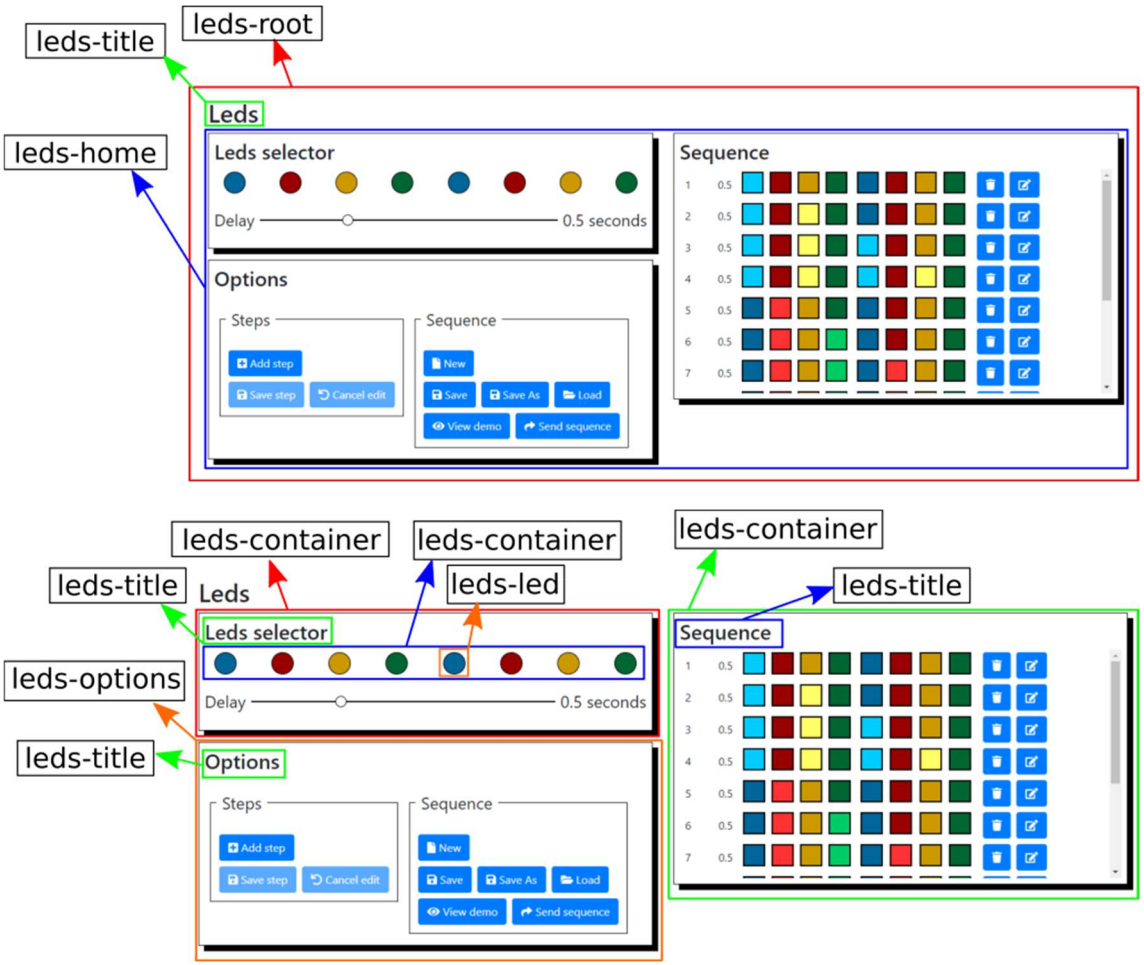
- **leds-title**: para mostrar el título de la pantalla.

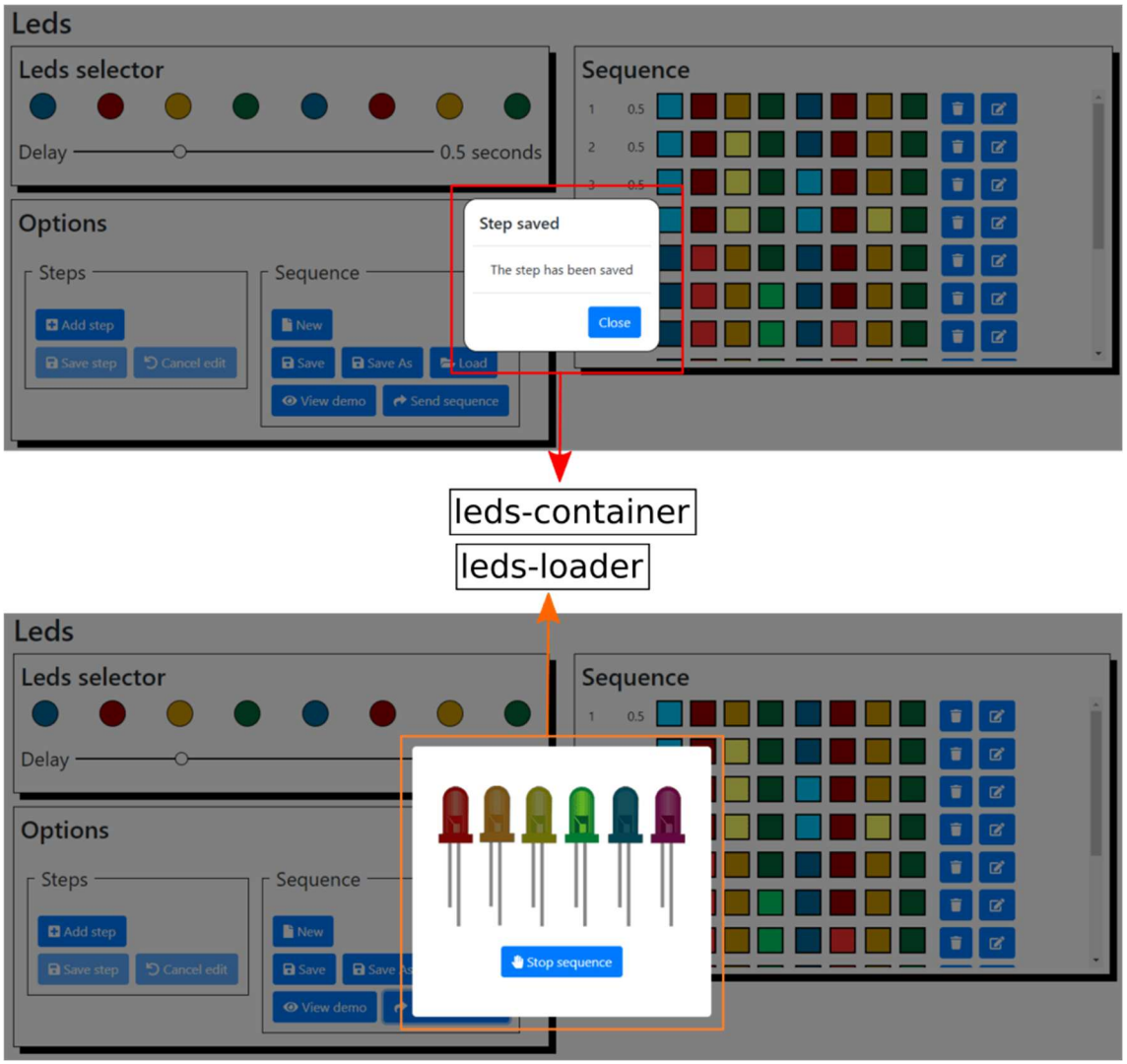
leds-messages-box

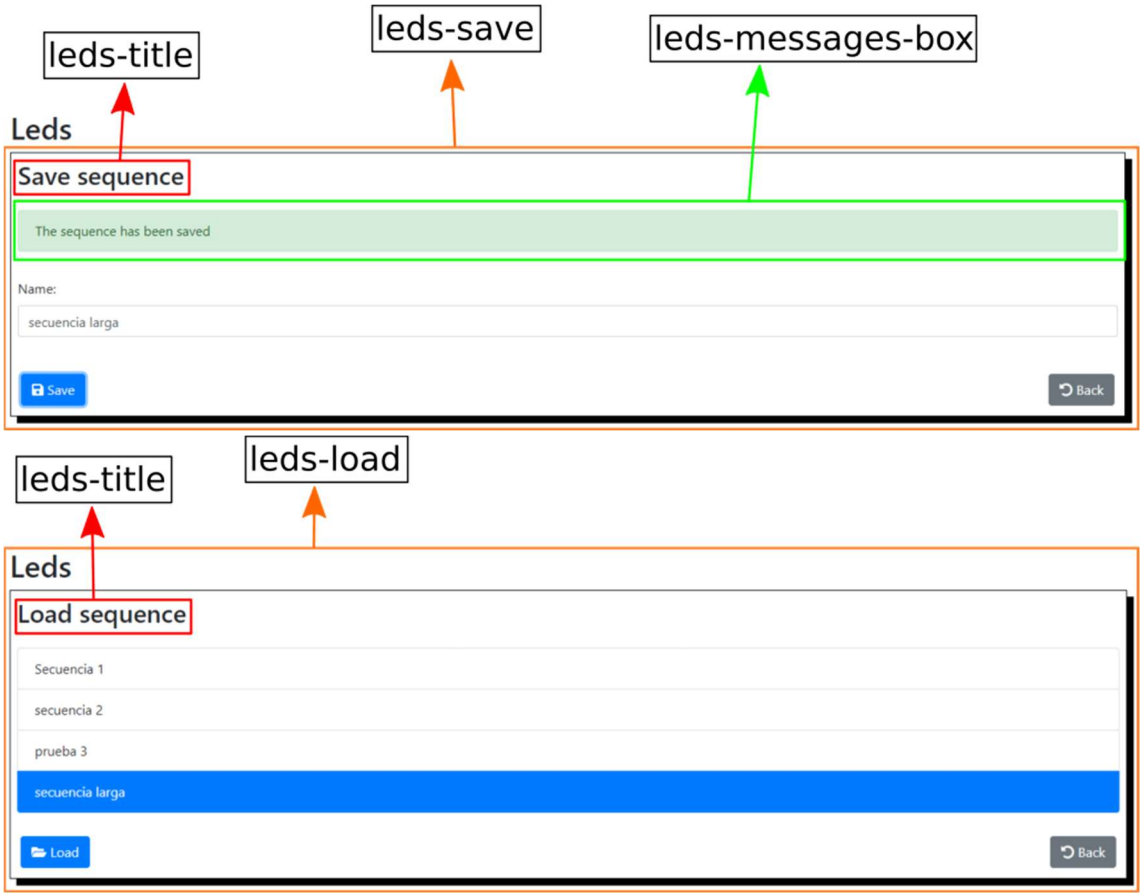
Este componente muestra mensajes sobre las acciones realizadas en la pantalla para guardar secuencias. Recibe dos parámetros:

- **type**: el tipo de mensaje que se ha de mostrar, pudiendo ser de error o de éxito.
- **message**: el mensaje que se ha de mostrar.
-

Localización de los componentes




Leds
Leds selector
Delay
0.5 seconds
Sequence
Options
Steps
Add step
Save step
Cancel edit
Sequence
New
Save
Save As
Load
View demo
Send sequence
Step saved
The step has been saved
Close
leds-container
leds-loader
Leds
Leds selector
Delay
Sequence
Options
Steps
Add step
Save step
Cancel edit
Sequence
New
Save
View demo
Stop sequence



Rutas

La aplicación maneja cuatro rutas:

- **home**: es la encargada de mostrar el componente leds-home, también es la ruta que se carga por defecto.
- **demos**: es la encargada de mostrar el componente leds-demo-player.
- **save**: es la encargada de mostrar el componente leds-save.
- **load**: es la encargada de mostrar el componente leds-load.

Interfaces

Led

Esta interfaz define un led dentro de un paso de la secuencia. Está formada por:

- **index (number)**: el índice del led dentro de la lista que forma el paso.
- **color (string)**: el color que muestra el led.
- **state (number)**: estado en el que está el led, encendido (1) o apagado (0).

LedState

Esta interfaz se usa para cambiar el estado en un led. Está formada por:

- **index (number)**: el índice del led dentro de la lista que forma el paso.
- **state (number)**: estado en el que está el led, encendido (1) o apagado (0).

Step

Esta interfaz se usa para almacenar un paso de la secuencia. Está formada por:

- **steps (Array<Led>)**: una lista de interfaces de led (8 en este caso), que forman un paso.
- **delay (number)**: el tiempo en que se mantiene en el estado el paso.

StateItem

Esta interfaz se usa para el envío de los datos de la secuencia a la Raspberry. Está formada por:

- **states (Array<number>)**: una lista con los estados de los leds, 0 o 1, según estén apagados o encendidos.
- **sleep (number)**: el tiempo en que se mantiene en el estado el paso.

OperationError

Esta interfaz se usa para gestionar los datos de los mensajes a mostrar al usuario al realizar operaciones con la secuencia. Está formada por:

- **error (boolean)**: indica si el mensaje es de error o no.
- **message (string)**: el mensaje a mostrar.

Servicios

Sequence

Este servicio se encarga de la gestión de las secuencias. Editar, guardar, cargar y enviar a la Raspberry. Dispone de los siguientes métodos:

- **setOperationError**: establece los valores para informar al usuario sobre el resultado de una operación.
- **setName**: establece el nombre de una secuencia para guardarla.
- **getOperationError**: recupera los valores para informar al usuario sobre el resultado de una operación.
- **getName**: recupera el nombre de una secuencia guardada.
- **getCurrent**: recupera la información del paso que se está editando.
- **getDelay**: recupera el valor de tiempo de espera para el paso actual.
- **getSteps**: recupera la lista de pasos de la secuencia actual.
- **getTotalSteps**: recupera el total de pasos de la secuencia actual.
- **getIdxStepEdit**: recupera el índice en la secuencia del paso que se esta editando.
- **getListSequencesSaved**: recupera una lista de todos los identificadores de la lista de secuencias guardadas en el local storage.
- **getStoredSequenceName**: recupera el nombre de una secuencia guardada en el local storage.
- **getSendSequenceClient**: recupera la promesa creada al enviar la secuencia a la Raspberry.
- **newSequence**: inicia una nueva secuencia.
- **resetSequence**: reinicia los valores para crear una nueva secuencia.
- **resetOperationError**: reinicia los valores del objeto usado para mostrar mensajes al usuario.
- **sendSequenceToServer**: envía la secuencia a la Raspberry para reproducir.

- **stopSequenceInServer**: envía la orden a la Raspberry para parar la reproducción.
- **saveSequence**: guarda la secuencia en el local storage.
- **loadStoreSequence**: carga una secuencia del local storage.
- **updateCurrentStep**: actualiza los valores del paso actual con los datos que recibe como parámetro.
- **sendCurrentStep**: envía un evento para indicar que los valores del paso actual se han modificado.
- **addCurrentStep**: añade los valores del paso actual a la secuencia.
- **saveStep**: guarda los datos de un paso cuando se está editando.
- **deleteStep**: borrar un paso de la secuencia.
- **editStep**: inicia la acción de editar un paso.
- **cancelEditStep**: cancela la acción de editar un paso.
- **formatNameToKey**: formatea el nombre de una secuencia para usarlo como clave y guardarla en el local storage.
- **stepForCopy**: serializa la información de un paso para añadirlo a la secuencia o guardarlo al editar.

Storage

Este servicio se encarga de la gestión del local storage del navegador. Dispone de los siguientes métodos:

- **store**: permite elegir que tipo de storage se va a usar, LOCAL o SESSION.
- **setValue**: guarda un valor en el storage.
- **getValue**: recupera un valor del storage.
- **keyExists**: comprueba si una clave o valor existe en el storage.
- **deleteValue**: elimina un valor del storage.
- **getJSONValue**: recupera un valor del storage en formato JSON.
- **setJSONValue**: guarda un objeto, con formato JSON, en el storage.

Repositorio y arranque de la aplicación

Para ejecutarla basta con clonar el proyecto mediante:

```
git clone https://github.com/theguitxo/remote-leds-client.git
```

y, dentro del directorio de la aplicación, ejecutar:

```
ng serve
```

y navegar a:

```
http://localhost:4200
```